SOFTWARE ENGINEERING LABORATORY

LAB–3

CLASS IDENTIFICATION, CLASSIFICATION AND CLASS DIAGRAM

LIBRARY MANAGEMENT SYSTEM

Submitted By	Ganga Tripathi
Enrollment Number	202401100200141
Course	B.Tech – Computer Science and Engineering
Subject	Software Engineering
Assignment	Lab–3: Identify and Classify Classes; Draw Class Diagram
Submitted To	Navdesh Singh

Object-Oriented Analysis and Design

1. Aim

To identify the major classes required for a Library Management System, classify them as strong and weak classes, and represent their attributes, operations, and relationships using a UML class diagram.

2. Introduction

A class is a blueprint used to create objects. It defines the data (attributes) and behaviour (methods) that its objects will have. In a Library Management System, classes represent entities such as users, books, loan transactions, and fines.

For this assignment, a **strong class** represents an entity that can exist independently in the system. A **weak class** represents a dependent entity whose meaningful existence is tied to one or more owner records. This is a modelling choice for the proposed system; in an implementation, weak records may still have their own generated IDs.

3. Identification and Classification of Classes

Class	Classification	Reason
User	Strong	Base user identity can exist independently and provides common account information.
Student	Strong (specialization of User)	Represents a registered library member with its own student-related details.
Librarian	Strong (specialization of User)	Represents a library employee who performs issue and return operations.
Administrator	Strong (specialization of User)	Represents a privileged system operator who manages accounts and system activities.
Book	Strong	A book catalogue record exists independently of any particular borrowing transaction.
LoanRecord	Weak	A loan record exists only when a student borrows a specific book; it depends on Student and Book.
Fine	Weak	A fine record exists only when it is associated with a loan record that incurs a charge.

4. Strong Classes

1. User (abstract class) – Stores common information shared by all system users.

Attributes: userId, name, email, passwordHash.
Methods: login(), logout().

2. Student – Represents a student who uses the library services.

Attributes: studentId, course.
Methods: searchBook(), viewHistory().

3. Librarian – Represents a library staff member who manages book circulation.

Attributes: employeeId.
Methods: issueBook(), acceptReturn().

4. Administrator – Represents a user with system-level permissions.

Attributes: adminId.
Methods: manageUsers(), generateReport().

5. Book – Represents a book available in the library catalogue.

Attributes: bookId, ISBN, title, author, availableQty.
Methods: checkAvailability(), updateDetails().

5. Weak Classes

1. LoanRecord – Represents the borrowing of a book by a student. It cannot be meaningful without the related Student and Book records.

Attributes: loanId, issueDate, dueDate, returnDate.
Methods: createLoan(), closeLoan().

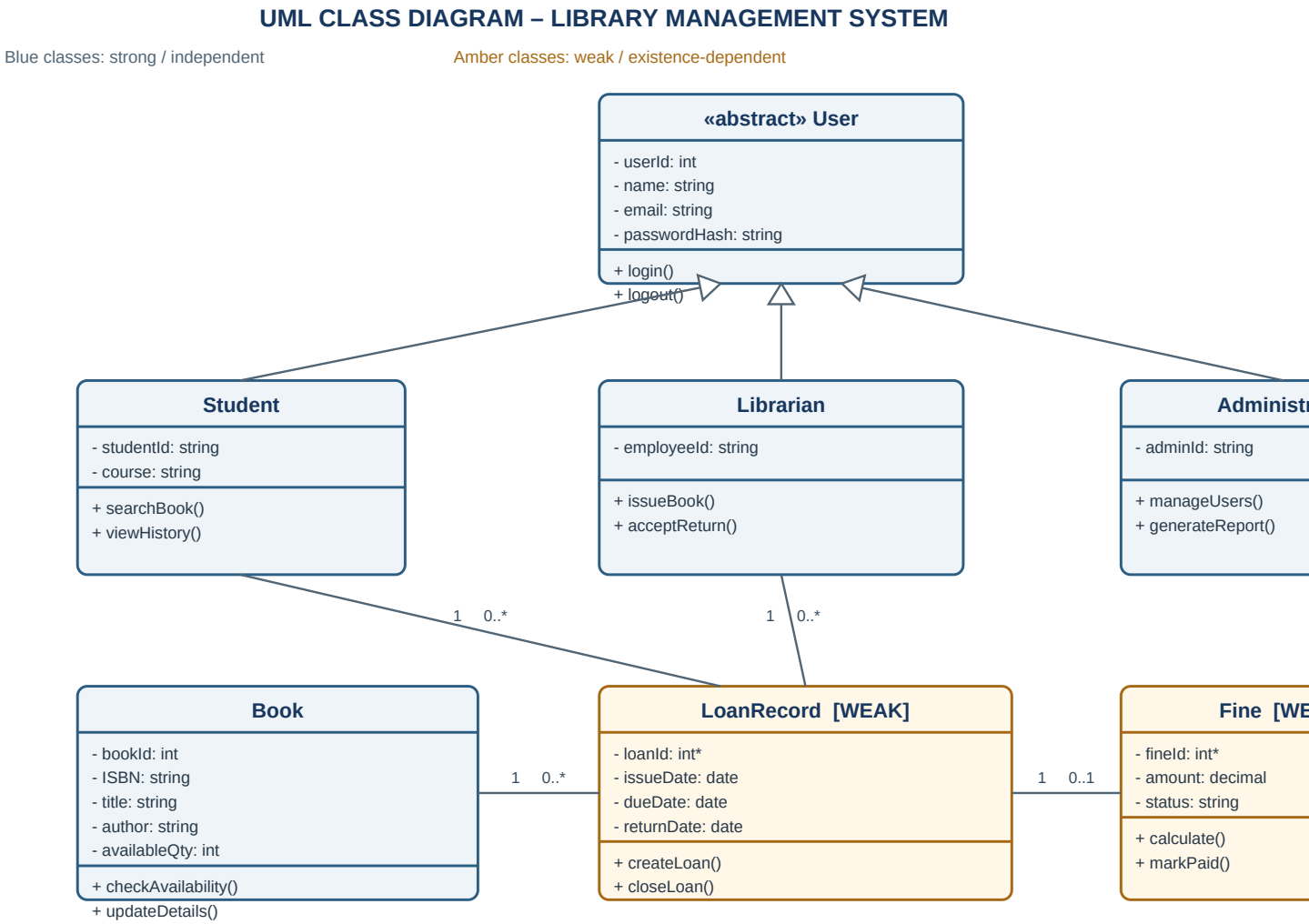
2. Fine – Represents an overdue charge for a loan. It depends on the related LoanRecord.

Attributes: fineId, amount, status.
Methods: calculate(), markPaid().

Note: The identifiers of weak classes (loanId and fineId) may be generated by the application for convenient database referencing. Their classification as weak is based on their lifecycle dependency, not on whether they have a technical primary key.

6. UML Class Diagram

The diagram shows inheritance from the abstract User class, associations between Student, Librarian, Book and LoanRecord, and the dependency of Fine on LoanRecord. Strong classes are shown in blue and weak classes in amber.



** IDs are shown as identifiers; weak classes are existence-dependent on their parent records.*

7. Explanation of Relationships

Relationship	Meaning
User → Student / Librarian / Administrator	Generalization (inheritance): these three classes inherit common attributes and operations from User.
Student — LoanRecord	One student can have zero or many loan records; each loan record belongs to one student.
Book — LoanRecord	One book title/record can be referenced by zero or many loan records over time; each loan record refers to one book record.
Librarian — LoanRecord	A librarian can process many loan records; a loan record is processed by a librarian.
LoanRecord — Fine	A loan record may have zero or one fine in this simplified model; each fine belongs to one loan record.

8. Conclusion

The Library Management System can be modelled using five strong classes—User, Student, Librarian, Administrator, and Book—and two weak classes—LoanRecord and Fine. The class diagram represents their attributes, methods, inheritance, and associations. This model provides a foundation for designing the system's object-oriented structure and implementing its database and application logic.

END OF LAB–3